ContactUsTest.java

```
package Footer;

import java.time.Duration;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.WebDriverWait;

import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class ContactUsTest {

   WebDriver driver;
   WebDriverWait wait;
   ContactUs contactForm;

   String url = "https://palakkad.lulumall.in/contact-us/";


   @BeforeMethod
   public void setUp() {

      driver = new ChromeDriver();
      driver.manage().window().maximize();

      wait = new WebDriverWait(driver, Duration.ofSeconds(30));

      driver.get(url);

      contactForm = new ContactUs(driver);

      System.out.println("Browser started - Contact Us page loaded");
   }


   // ============================================================
   // DATA PROVIDER: Wide range of input combinations
   // Columns: testCase, firstName, lastName, email, phone,
   //          queryType, message, expectFormValid
   // ============================================================

   @DataProvider(name = "contactFormData")
   public Object[][] contactFormData() {

      return new Object[][] {

    /*     // 1. Valid input - expected pass
         { "01 - Valid normal input", "John", "Doe", "john.doe@example.com",
           "9876543210", "Request or Query", "This is a valid message.", true, true },
```

```
        // 2. Invalid input - expected fail (generic garbage input across all fields)
        { "02 - Invalid input across fields", "!!!", "###", "not-an-email",
          "abc", "Request or Query", "msg", false, false },

        // 3. Valid lowercase - expected pass
        { "03 - Valid lowercase", "john", "doe", "john@example.com",
          "9000000000", "Promotion/Event", "lowercase message", true, true },

        // 4. Valid UPPERCASE - expected pass
        { "04 - Valid UPPERCASE", "JOHN", "DOE", "JOHN@EXAMPLE.COM",
          "9999999999", "Request or Query", "UPPERCASE MESSAGE", true, true },

        // 5. Name with special chars/space/dot/numbers - expected FAIL (business rule, not native
HTML5)
        { "05 - Name with special chars/numbers/dots", "John123.@#", "Doe",
          "john@example.com", "9876543210", "Request or Query", "Message", false, true },

        // 6. Mixed case name - expected pass
        { "06 - Mixed case name", "JoHn", "DoE", "JoHn.DoE@Example.Com",
          "9812345678", "Customer Feedback", "MiXeD CaSe MeSsAge", true, true },

        // 7. All fields empty - expect required message shown
        { "07 - All fields empty", "", "", "", "", "", "", false, false },

        // 8a. Only first name empty
        { "08a - First name empty only", "", "Doe", "john@example.com",
          "9876543210", "Request or Query", "Message", false, false },

        // 8b. Only last name empty
        { "08b - Last name empty only", "John", "", "john@example.com",
          "9876543210", "Request or Query", "Message", false, false },

        // 9. First name filled, last name empty (duplicate-style check of #8b, kept per your list)
        { "09 - First name with last name empty", "John", "", "john@example.com",
          "9876543210", "Request or Query", "Message", false, false },

        // 10. First name contains space (e.g. "John Doe" as one field) - expected pass
        { "10 - First name contains space", "John Doe", "Smith",
          "john@example.com", "9876543210", "Request or Query", "Message", true, true },

        // 11. Last name without first name - expected fail / required msg on first name
        { "11 - Last name without first name", "", "Smith",
          "john@example.com", "9876543210", "Request or Query", "Message", false, false },

        // 12. First name starts with space or other char - expected fail
        { "12a - First name starts with space", " John", "Doe",
          "john@example.com", "9876543210", "Request or Query", "Message", false, true },

        { "12b - First name starts with symbol", "@John", "Doe",
          "john@example.com", "9876543210", "Request or Query", "Message", false, true },

        // 13. Last name starts with space or other char - expected fail
        { "13a - Last name starts with space", "John", " Doe",
          "john@example.com", "9876543210", "Request or Query", "Message", false, true },

        { "13b - Last name starts with symbol", "John", "#Doe",
          "john@example.com", "9876543210", "Request or Query", "Message", false, true },
```

```
        // 14. Names with single characters - expected pass
        { "14 - Single character names", "J", "D",
          "j@example.com", "9876543210", "Request or Query", "Message", true, true },

        // 15. Very long name - expected pass (no maxlength set)
        { "15 - Very long name", "Johnathaniel Alexander Maximilian",
          "Wolfeschlegelsteinhausenbergerdorff", "long.name@example.com",
          "9876543210", "Request or Query", "Message", true, true },

        // 16. Valid email - expected pass
        { "16 - Valid email", "John", "Doe", "john.doe@example.com",
          "9876543210", "Request or Query", "Message", true, true },

        // 17. Invalid email - expected fail
        { "17 - Invalid email (no @)", "John", "Doe", "john.example.com",
          "9876543210", "Request or Query", "Message", false, false },

        // 18a. Email with leading dot in local part - expected fail
        { "18a - Email leading dot", "John", "Doe", ".john@example.com",
          "9876543210", "Request or Query", "Message", true, false },

        // 18b. Email with trailing dot before @ - expected fail
        { "18b - Email trailing dot", "John", "Doe", "john.@example.com",
          "9876543210", "Request or Query", "Message", true, false },                    */

        // 18c. Email with consecutive dots - expected fail
        { "18c - Email consecutive dots", "John", "Doe", "jo..hn@example.com",
          "9876543210", "Request or Query", "Message", true, false },

    /*      // 18d. Email local part exceeding 64 chars - expected fail
        { "18d - Email local part too long", "John", "Doe",
          "a".repeat(65) + "@example.com",
          "9876543210", "Request or Query", "Message", true, false },

        // 18e. Email with disallowed special character - expected fail
        { "18e - Email disallowed character", "John", "Doe", "john#doe@example.com",
          "9876543210", "Request or Query", "Message", true, false },                    */

        // 19. Plus-tag email - expected pass
        { "19 - Email with plus tag", "John", "Doe", "john+test@example.com",
          "9876543210", "Request or Query", "Message", true, true },

    /*      // 20. Phone less than 10 digits - expected fail
        { "20 - Phone less than 10 digits", "John", "Doe", "john@example.com",
          "98765", "Request or Query", "Message", false, false },                */

        // 21. Phone more than 10 digits - expected fail
        { "21 - Phone more than 10 digits", "John", "Doe", "john@example.com",
          "987654321099", "Request or Query", "Message", false, false },

    /*      // 22. Phone with letters - expected fail
        { "22 - Phone with letters", "John", "Doe", "john@example.com",
          "98765abcde", "Request or Query", "Message", false, false },

        // 23. Phone with symbols/dashes - expected fail
        { "23 - Phone with symbols/dashes", "John", "Doe", "john@example.com",
```

```
  "987-654-3210", "Request or Query", "Message", false, false },

 // 24. Phone with spaces - expected fail
 { "24 - Phone with spaces", "John", "Doe", "john@example.com",
  "98765 43210", "Request or Query", "Message", false, false },

 // 25. Phone starting with 0 - expected pass (still 10 digits, pattern allows any digit)
 { "25 - Phone starting with 0", "John", "Doe", "john@example.com",
  "0876543210", "Request or Query", "Message", true, true },

 // 26. Dropdown left blank - expected fail
 { "26 - Dropdown left blank", "John", "Doe", "john@example.com",
  "9876543210", "", "Message", false, false },

 // 27. Query message very long
 { "27 - Very long query message", "John", "Doe", "john@example.com",
  "9876543210", "Request or Query", "This is a very long message. ".repeat(30),
  true, true },

 // 28. Message with special characters - expected pass (textarea has no restriction)
 { "28 - Message with special characters", "John", "Doe", "john@example.com",
  "9876543210", "Request or Query", "!@#$%^&*()_+-=[]{}|;':\",./<>?",
  true, true },

 // -------- Additional validations worth adding --------                    */

 // 29. Email exactly 64 chars local part - boundary case, should pass
 { "29 - Email local part exactly 64 chars", "John", "Doe",
  "a".repeat(64) + "@example.com",
  "9876543210", "Request or Query", "Message", true, true },

 /*       // 30. Phone with unicode/full-width digits - expected fail
 { "30 - Phone with unicode digits", "John", "Doe", "john@example.com",
  "９８７６５４３２１０", "Request or Query", "Message", false, false },

 // 31. Name with tab/newline character injected - expected fail
 { "31 - Name with newline character", "John\n", "Doe",
  "john@example.com", "9876543210", "Request or Query", "Message", false, true },

 // 32. Basic script/HTML injection attempt in message - should not break, treated as plain text
 { "32 - Message with script injection attempt", "John", "Doe",
  "john@example.com", "9876543210", "Request or Query",
  "<script>alert('xss')</script>", true, true },                              */

 // 33. Basic SQL injection attempt in name - business rule should reject (has symbols)
 { "33 - Name with SQL injection attempt", "John' OR '1'='1", "Doe",
  "john@example.com", "9876543210", "Request or Query", "Message", false, true },

/*       // 34. Email with international domain (should generally be treated as valid syntax)
 { "34 - Email with country-code TLD", "John", "Doe", "john@example.co.uk",
  "9876543210", "Request or Query", "Message", true, true },

 // 35. Trailing/leading spaces around entire email address
 { "35 - Email with surrounding spaces", "John", "Doe", "  john@example.com  ",
  "9876543210", "Request or Query", "Message", false, false },                 */

 // 36. All three name/email/phone valid but message left as only whitespace
```

```
            { "36 - Message whitespace only", "John", "Doe", "john@example.com",
              "9876543210", "Request or Query", "     ", true, false },
        };
    }


    // ============================================================
    // TEST: Fill form, attempt submit, verify client-side validity
    // ============================================================

    @Test(dataProvider = "contactFormData")
    public void testContactFormValidation(
            String testCase,
            String firstName,
            String lastName,
            String email,
            String phone,
            String queryType,
            String message,
            boolean expectedHtml5Valid,
            boolean expectedBusinessValid) {

        System.out.println("================================================");
        System.out.println("Test Case: " + testCase);

        contactForm.fillForm(firstName, lastName, email, phone, queryType, message);

        // Layer 1: native HTML5 constraint validation (what the browser currently enforces)
        boolean html5Valid =
                contactForm.isFieldValid(contactForm.getFirstNameLocator())
                && contactForm.isFieldValid(contactForm.getLastNameLocator())
                && contactForm.isFieldValid(contactForm.getEmailLocator())
                && contactForm.isFieldValid(contactForm.getPhoneLocator())
                && contactForm.isFieldValid(contactForm.getMessageLocator())
                && (queryType != null && !queryType.isEmpty());

        // Layer 2: your business rules (letters-only names, strict RFC-lite email)
        boolean businessValid =
                contactForm.isNameFormatValid(firstName.trim())
                && contactForm.isNameFormatValid(lastName.trim())
                && contactForm.isEmailFormatValid(email.trim())
                && phone.matches("\\d{10}")
                && !message.trim().isEmpty()
                && (queryType != null && !queryType.isEmpty());

        System.out.println("HTML5 valid -> expected: " + expectedHtml5Valid + " | actual: " + html5Valid);
        System.out.println("Business rule valid -> expected: " + expectedBusinessValid + " | actual: " +
businessValid);

        Assert.assertEquals(html5Valid, expectedHtml5Valid,
                "HTML5 validation mismatch for: " + testCase);

        Assert.assertEquals(businessValid, expectedBusinessValid,
                "Business rule validation mismatch for: " + testCase);

        System.out.println(testCase + " -> PASSED");
    }
```

```
    @AfterMethod
    public void tearDown() {

        if (driver != null) {
            driver.quit();
            driver = null;
        }

        System.out.println("Browser closed");
    }
}
```